Lab 09: Socket Programming

Duration: 2 hours | In Lab Evaluation & Assignment
Tools: Python 3 (`socket`)

Objectives

By the end of this lab, students will be able to:

- Describe UDP's connectionless, unreliable datagram service.
- Implement a UDP echo server and client in Python, with error handling, timeouts, retries, and basic concurrency.
- Use Python's `logging` module to capture runtime events.
- Explain the TCP three-way handshake and connection teardown.
- Write a TCP echo server and client in Python using `SOCK_STREAM` sockets.
- Demonstrate persistence of a TCP connection across multiple send/receive calls.
- Observe TCP connection states using `netstat` or `ss`.
- Configure inter-PC communication within the lab network and parameterize client/server addresses.

Lab Description

Part 0: Installing Python

1. **Windows:**
   (a) Download the latest Windows installer from python.org.
   (b) Run the installer, ensure `Add Python to PATH` is checked, and click `Install Now`.
   (c) Verify in Command Prompt:

```
python --version
pip --version
```

2. **macOS:**
   (a) Open Terminal. If Python3 is missing, install Homebrew from brew.sh, then run:

```
brew install python
```

   (b) Verify:

```
python3 --version
pip3 --version
```

3. **Linux (Ubuntu/Debian):**
   (a) Update package lists:

```
sudo apt update
```

(b) Install Python 3 and pip:

```
sudo apt install python3 python3-pip
```

(c) Verify:

```
python3 --version
pip3 --version
```

4. You are now ready to use Python for scripting and automation within your lab exercises.

Part A: UDP Echo Server

1. In `udp_server.py`, import socket and configure the server:

```python
# Import the socket module to create UDP sockets
import socket

# Create a UDP socket (AF_INET = IPv4, SOCK_DGRAM = UDP)
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)


# Bind the socket to all interfaces on port 5005
# '0.0.0.0' makes the server reachable from any network its
    attached to

sock.bind(('0.0.0.0', 5005))
print("UDP server listening on port 5005...")
```

2. Implement the receive-and-echo loop with error handling:

```python
try:
    # Enter an infinite loop to receive and echo packets
    while True:
        # recvfrom(1024) waits for up to 1024 bytes from any
            client
        data, addr = sock.recvfrom(1024)
        # Print the raw data and the client address (IP, port)
        print(f"Received {data!r} from {addr}")
        # Send the exact same data back to the sender
        sock.sendto(data, addr)
except KeyboardInterrupt:
    print("\nShutting down server.")
    sock.close()
```

Part B: UDP Client

1. In `udp_client.py`, import modules and set up the socket:

```python
# Import socket for networking and time for RTT measurements
```

```
import socket, time

# Create a UDP socket
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# Set a 1 second timeout for blocking socket operations
# This is the time which we will wait for the echo
sock.settimeout(1.0)

# Define server address as localhost on port 5005
# localhost -> own machine -> 127.0.0.1
server_addr = ('localhost', 5005)
```

2. Send a single message and receive echo:

```
#---- Single message exchange ----
message = 'Hello, UDP Server'
# Encode the string to bytes and send to the server
sock.sendto(message.encode('utf-8'), server_addr)
try:
    # Wait for up to 1024 bytes response
    data, _ = sock.recvfrom(1024)
    print("Echo:", data)
except socket.timeout:
    print("No response, server may be down.")
```

3. Extend to multiple messages with retry:

```
#---- Multiple messages with retry and RTT ----
rtts = []  # to store round-trip times

# sending 5 messages with max 3 retries
for i in range(5):
    text = f"Msg {i}".encode('utf-8')
    attempts = 0
    while attempts < 3:
        start = time.time()                # start RTT timer
        sock.sendto(text, server_addr)     # send datagram
        try:
            data, _ = sock.recvfrom(1024) # wait for echo
            elapsed = time.time() - start
            rtts.append(elapsed)
            print(f"Response {i}:", data, f"(RTT={elapsed:.3f}s)")
            break
        except socket.timeout:
            attempts += 1
            print(f"Retry {attempts} for message {i}")
    time.sleep(0.5)  # brief pause between messages
```

Part C: Testing and Analysis

1. **Run Server:**

- Terminal 1: `python3 udp_server.py`

2. **Run Client:**
   - Terminal 2: `python3 udp_client.py`
3. Observe:
   - Server console logs each datagram and client address.
   - Client console shows echoes or timeout/retry messages.

Part D: Concurrency

Till now, the server would have processed the messages from more than one clients (if there were multiple clients) in a sequential manner. So, if multiple clients send messages to the server, it can't process them at the same time. To make it happen, we have to use threading. In simple terms, threading gives us the ability to divide the process (in our case the server.py) into multiple threads (different methods can be different threads) which can run concurrently. You can find details in this link - Multi-threading. In our case, we can use handle requests as a thread so that if there are multiple clients, there can be multiple threads handling those clients at the same time. Below is the modified server code to handle multiple clients at the same time -

```
import socket
import threading

# Create UDP server socket
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
# Allow address reuse
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

sock.bind(('0.0.0.0', 5005))
print("UDP server listening on port 5005...")

# Handler for each incoming message
def handle_request(data, addr):
    print(f"Received {data!r} from {addr}")
    sock.sendto(data, addr)

try:
    while True:
        data, addr = sock.recvfrom(1024)
        # Spawn a new thread per request for concurrent handling
        # target specifies the callable (function or method) that the
            new thread will execute
        # A tuple of positional arguments to pass into target. In this
            case, once the thread begins, it effectively does:
            handle_request (data, addr)
        # daemon  ensures threads wont prevent program exit

        threading.Thread(target=handle_request, args=(data, addr),
            daemon=True).start()
except KeyboardInterrupt:
    print("\nShutting down server.")
```

```
28      sock.close()
```

Part E: Logging in python

The logging module allows you to capture runtime events with timestamps, severity levels, and flexible output destinations.

- Basic configuration

```
import logging
logging.basicConfig(
  filename='server.log',
  level=logging.INFO,
  format='%(asctime)s - %(message)s',
  datefmt='%Y-%m-%d %H:%M:%S'
)
```

- Writing log entries

```
logging.info("Alice registered from ('127.0.0.1', 54321)")
logging.error("Failed to forward message")
```

- Use debug(), info(), warning(), error(), critical() for different severities.

Part F: TCP Echo Server

1. Create a new Python file `tcp_echo_server.py`.
2. In the file, import the socket module:

```
import socket
```

3. Build a TCP server socket that:
   - Binds to all interfaces on port 6006.
   - Listens for a single incoming connection.
   - Accepts the connection and prints the client's address.
   - In a loop, receives up to 1024 bytes, echoes it back, and prints each message.
   - Closes the connection cleanly when the client disconnects.
4. Example skeleton:

```
# --- SERVER SIDE ---

# 1. Create a new socket object using IPv4 and TCP
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 2. Bind the socket to all network interfaces ('0.0.0.0') on port
    6006
server.bind(('0.0.0.0', 6006))

# 3. Put the socket into listening mode, with a backlog of 5
   queued connection
```

```
server.listen(5)

# 4. Block and wait for an incoming connection; when one arrives,
#    accept() returns a new socket object (conn) and the client
   address (addr)
conn, addr = server.accept()

# 5. Print the address of the newly connected client
print("Connected by", addr)

# 6. Enter an infinite loop to continually receive and echo data
while True:
    # 7. Read up to 1024 bytes from the client; blocks until data
       arrives
    data = conn.recv(1024)

    # 8. If no data was received, it means the client closed the
       connection - break out
    if not data:
        break

    # 9. Print what was received for debugging/logging
    print("Received:", data)

    # 10. Send the exact same bytes back to the client
    conn.sendall(data)

# 11. Close the client socket once done
conn.close()
```

5. Save and run the server:

   ```
   $ python3 tcp_echo_server.py
   ```

Part G: TCP Client

1. Create `tcp_echo_client.py`.
2. In the file, import `socket` and:
   - Create a TCP socket.
   - Connect to `localhost` on port 6006.
   - Send a greeting (e.g. `b"Hello, TCP"`).
   - Receive and print the echoed data.
3. Example snippet:

```
# --- CLIENT SIDE ---

# 1. Create a new socket object using IPv4 and TCP
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

```

```
# 2. Connect this socket to the server running on localhost port 6006
client.connect(('127.0.0.1', 6006))

# 3. Loop over a list of bytestrings to send
for msg in [b"Hello, TCP!", b"Message 2", b"Final msg"]:
    #   a) Send the entire message to the server (blocks until sent)
    client.sendall(msg)

    #   b) Wait for up to 1024 bytes of response from the server (the echo)
    data = client.recv(1024)

    #   c) Print out what the server echoed back
    print("Echo:", data)

# 4. Close the client socket when all messages have been sent and echoed
input("Press ENTER here to close the connection and exit...")
client.close()
```

4. Run the client while the server is listening.

Part H: Communicating within the lab

As you already know, each of our lab PCs are connected with one another through a central switch. To make one PC act as the TCP echo server and all others as clients, follow these detailed steps:

1. **Choose the Server Machine and Assign IP Address**
   (a) On the designated server PC, open a terminal (Linux/macOS) or Command Prompt (Windows).
   (b) Check its current IP address:
   - **Windows:** `ipconfig`
   - **Linux/macOS:** `ip addr show` or `ifconfig`

   (c) If the network uses DHCP, note the assigned IP (e.g. `192.168.1.10`). Otherwise, set a static IP in the same subnet as the clients (e.g. `192.168.1.10/24`).
2. **Configure Firewall on server machine to Allow TCP Port 6006**
   (a) **Windows:**
   - Open *Windows Defender Firewall then go to Advanced Settings.*
   - Create a new *Inbound Rule* allowing TCP port `6006`.

   (b) **Linux (using `ufw`):**

```
sudo ufw allow 6006/tcp
sudo ufw reload
```

3. **Run the TCP Echo Server on the Server PC**
   (a) Ensure `tcp_echo_server.py` is present.
   (b) Launch the server:

```
$ python3 tcp_echo_server.py
```

   (c) The server will bind to `0.0.0.0:6006` and wait for incoming connections.
4. **Update and Run Clients on Each Student PC**
   (a) On every client PC, open `tcp_echo_client.py` in a text editor.
   (b) Modify the `connect()` line to point at the server's IP:

```
client.connect(('192.168.1.10', 6006))
```

   (Replace `192.168.1.10` with your actual server IP.)
   (c) Save and run the client:

```
$ python3 tcp_echo_client.py
```

   (d) Observe each client printing three `Echo: ...` lines.
5. **Optional: Parameterize Server IP for Flexibility**
   (a) Edit `tcp_echo_client.py` to accept the server IP as a command-line argument:

```
import sys, socket

server_ip = sys.argv[1]       # Usage: python3 tcp_echo_client.
    py 192.168.1.10
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((server_ip, 6006))
```

   (b) Run with:

```
$ python3 tcp_echo_client.py 192.168.1.10
```